Dev Sprint’26

DevOps Hackathon 2026

Final List of 35 Problem Statements

Common Rules for All Participants

Event schedule

- 8:45–9:15 AM: Registration and environment verification.
- 9:15–9:30 AM: Problem statement selection.
- 9:30 AM–12:20 PM: Development, testing, documentation and deployment preparation.
- 12:20–1:30 PM: Lunch break.
- 1:30–2:10 PM: Deployment and verification.
- 2:10–2:30 PM: Final submission and demonstration readiness.

Team size: Minimum 2 and maximum 3 members.

Mandatory requirements:

1. Build a working prototype that implements the primary functionality of the selected problem.
2. Deploy the solution to an externally accessible Linux server, VPS, cloud environment or approved self-hosted server.
3. Do not use GitHub Pages, Vercel, Netlify or similar ready-made static hosting platforms as the primary hosting solution.
4. Submit source code, a README, a simple architecture diagram and evidence of testing.
5. Demonstrate at least one meaningful feature of the solution.
6. Use a minimum viable implementation. Additional features are optional.

7. Prepare cloud accounts, credentials, required tools and deployment environments before the event.

Important: Teams may use local Docker containers, Docker Compose and predefined configurations where appropriate. Kubernetes and complex cloud infrastructure are not mandatory unless explicitly required by the selected challenge.

Category 1: CI/CD and Release Automation

These challenges focus on automating software builds, testing, deployments and release management.

Problem 1: Automated CI/CD Pipeline for Web Applications

Difficulty: Beginner–Intermediate

Problem Statement

Develop an automated CI/CD pipeline that builds, tests and deploys a sample web application whenever new code is pushed to a Git repository. The pipeline must stop deployment if the automated tests fail.

Minimum Requirements

- Create a simple web application.
- Configure a GitHub Actions workflow triggered by a code push.
- Execute at least one automated test.
- Build a Docker image.
- Deploy the application to an accessible server.
- Display pipeline execution logs.

Expected Deliverables

- Source repository and CI/CD workflow.

- Dockerfile and deployment configuration.
- Automated test results.
- Working deployed application.
- README documenting the workflow.

Final Demonstration

Modify the application, push the change, and demonstrate the automated testing and deployment process.

Suggested tools: GitHub Actions, Git, Docker, Nginx, Node.js or Python.

Problem 2: Automated Code Quality Gate

Difficulty: Beginner–Intermediate

Problem Statement

Develop a CI workflow that checks source code for basic quality issues before allowing it to proceed to deployment. The system must run automated tests and code-quality checks and reject code that violates the configured rules.

Minimum Requirements

- Create a sample application.
- Integrate a linter and automated tests.
- Configure a pass/fail quality gate.
- Include one intentionally faulty code change.
- Display the reason for a failed quality check.
- Deploy the application when all checks pass.

Expected Deliverables

- Source repository and CI workflow.
- Linting and test results.

- Quality-gate configuration.
- Evidence of a rejected faulty change.
- Working deployed application.

Final Demonstration

Push a valid change and then a faulty change. Show the pipeline passing the first and rejecting the second.

Suggested tools: GitHub Actions, ESLint or Ruff, Jest or pytest, Docker.

Problem 3: Zero-Downtime Deployment Simulator

Difficulty: Intermediate

Problem Statement

Implement a blue-green deployment system that switches between two application versions while minimizing service interruption. The system must verify the health of the new version before directing traffic to it.

Minimum Requirements

- Run two distinguishable application versions.
- Configure Nginx as a reverse proxy.
- Implement a script to switch between versions.
- Perform a health check before switching traffic.
- Retain the previous version if the new version is unhealthy.
- Record deployment and switching events.

Expected Deliverables

- Two application versions.
- Nginx configuration.
- Deployment and health-check scripts.

- Logs showing the switching process.
- Working deployed application.

Final Demonstration

Send repeated requests while switching versions. Demonstrate that requests continue succeeding and that an unhealthy version is not activated.

Suggested tools: Docker Compose, Nginx, Bash, curl.

Problem 4: Automated Release Rollback System

Difficulty: Intermediate

Problem Statement

Develop a deployment workflow that automatically restores the previous working application version when a newly deployed version fails its health checks.

Minimum Requirements

- Maintain two identifiable application versions.
- Implement version selection.
- Deploy the selected version.
- Check application health.
- Automatically restore the previous version after a failed health check.
- Record deployment and rollback events.

Expected Deliverables

- Versioned application images or releases.
- Deployment and rollback script.
- Health-check mechanism.
- Logs showing a failed release and successful rollback.
- README containing execution instructions.

Final Demonstration

Deploy an intentionally unhealthy version and demonstrate that the system detects the failure and restores the previous working version.

Suggested tools: Docker, Docker Compose, Bash, Python.

Problem 5: Software Build Artifact Registry

Difficulty: Intermediate

Problem Statement

Develop a workflow that generates versioned software artifacts, stores them in a designated repository, and verifies their integrity before deployment.

Minimum Requirements

- Build two distinguishable application versions.
- Assign a unique version or build identifier to each artifact.
- Store the artifacts in a registry or designated artifact repository.
- Generate SHA-256 checksums.
- Retrieve a selected artifact and verify its checksum.
- Document the artifact retrieval process.

Expected Deliverables

- Two versioned artifacts.
- Artifact storage or registry configuration.
- Build metadata and checksums.
- Verification script.
- Documentation and test evidence.

Final Demonstration

Publish two versions, retrieve one version, and verify that its checksum matches the recorded value.

Suggested tools: GitHub Actions, Docker Registry or GitHub Packages, SHA-256 utilities.

Category 2: Containers and Orchestration

These challenges focus on containerization, service recovery, resource management and container networking.

Problem 6: Containerize a Legacy Application

Difficulty: Beginner–Intermediate

Problem Statement

Package a sample application into a Docker container so that it can run consistently without manually installing its dependencies on the host machine.

Minimum Requirements

- Select a simple existing application.
- Create a Dockerfile.
- Configure application dependencies.
- Use environment variables for configuration.
- Build and run the Docker image.
- Make the application accessible through a browser or API client.

Expected Deliverables

- Source code and Dockerfile.
- Dependency configuration.
- Docker Compose file if needed.
- Working containerized application.
- README with build and execution instructions.

Final Demonstration

Build the image and run the application in a container. Show that it works without manually installing the application dependencies on the host.

Suggested tools: Docker, Docker Compose, Node.js or Python.

Problem 7: Self-Healing Container Platform

Difficulty: Intermediate

Problem Statement

Develop a containerized application that detects failures and automatically recovers from common service interruptions.

Minimum Requirements

- Containerize a sample application.
- Configure an application health endpoint.
- Define an appropriate container restart policy.
- Configure health checks.
- Demonstrate the difference between a running container and a healthy application.
- Record recovery events or status changes.

Expected Deliverables

- Dockerfile and container configuration.
- Health-check endpoint.
- Restart policy and health-check configuration.
- Evidence of recovery after a controlled failure.
- README describing the recovery process.

Final Demonstration

Trigger a controlled container failure and demonstrate the recovery process.

Suggested tools: Docker, Docker Compose, Python or Node.js.

Problem 8: Kubernetes Application Deployment and Scaling

Difficulty: Intermediate–Advanced

Problem Statement

Deploy a containerized application to a Kubernetes environment and demonstrate replica management, health checking and automatic replacement of failed pods.

Minimum Requirements

- Use a working Kubernetes environment.
- Deploy a containerized application.
- Create Deployment and Service manifests.
- Configure readiness and liveness probes.
- Scale the application from one to three replicas.
- Demonstrate replacement of a deleted pod.

Expected Deliverables

- Docker image.
- Kubernetes manifests.
- Deployment and scaling commands.
- Evidence of pod replacement.
- Working application endpoint.
- README with deployment instructions.

Final Demonstration

Deploy the application, scale it to three replicas, delete one pod and demonstrate Kubernetes creating a replacement.

Suggested tools: Kubernetes, Minikube or kind, Docker, kubectl.

Scope restriction: Teams must have a working local Kubernetes environment or access to a prepared cluster. Installing and configuring a new Kubernetes environment from scratch is not part of the challenge.

Problem 9: Container Resource Optimization

Difficulty: Intermediate

Problem Statement

Build a tool that monitors container CPU and memory usage and demonstrates how resource limits affect a container running a controlled workload.

Minimum Requirements

- Create a containerized sample workload.
- Collect CPU and memory usage.
- Display resource usage before optimization.
- Apply configurable resource limits.
- Repeat the workload and record the results.
- Generate a basic comparison report.

Expected Deliverables

- Sample workload and Docker configuration.
- Resource-monitoring script or dashboard.
- Configurable resource limits.
- Before-and-after measurements.
- Short report explaining the results.

Final Demonstration

Run the workload, observe its resource usage, apply limits and show the resulting behavior.

Suggested tools: Docker, Python, Docker statistics, cAdvisor or Grafana.

Problem 10: Container Network Isolation

Difficulty: Intermediate

Problem Statement

Create a multi-container application with restricted network communication so that each service can access only the components required for its operation.

Minimum Requirements

- Create a frontend, backend and database service.
- Configure Docker networks to restrict communication.
- Allow frontend-to-backend communication.
- Allow backend-to-database communication.
- Prevent direct frontend-to-database communication.
- Test permitted and prohibited connections.

Expected Deliverables

- Docker Compose configuration.
- Three services or equivalent test containers.
- Network diagram.
- Connectivity test results.
- Working deployed application.

Final Demonstration

Show that permitted connections succeed and prohibited connections fail.

Suggested tools: Docker Compose, Docker bridge networks, curl, netcat.

Category 3: Infrastructure as Code and Cloud

These challenges focus on infrastructure provisioning, server configuration, backups, configuration drift and cost estimation.

Problem 11: One-Command Infrastructure Provisioning

Difficulty: Intermediate

Problem Statement

Develop an Infrastructure as Code solution that provisions a small, repeatable application environment using declarative configuration.

Minimum Requirements

- Create Terraform configuration files.
- Define at least two related infrastructure resources.
- Use variables for configurable values.
- Generate and review a Terraform plan.
- Provision the environment.
- Demonstrate controlled cleanup.

Expected Deliverables

- Terraform configuration.
- Variables and outputs.
- Successful plan and deployment evidence.
- Resource or service endpoint information.
- README containing setup and cleanup instructions.

Final Demonstration

Provision the environment, show the created resources and demonstrate how a configuration change affects the Terraform plan.

Suggested tools: Terraform, Docker provider or an approved cloud provider.

Scope restriction: Local Docker resources or a prepared cloud sandbox are acceptable. Paid cloud provisioning is not mandatory.

Problem 12: Automated Linux Server Configuration

Difficulty: Beginner–Intermediate

Problem Statement

Automate the configuration of a Linux server using Ansible. The playbook must install a web server, configure a simple web page and ensure the service is running.

Minimum Requirements

- Create an Ansible inventory.
- Write a playbook to install Nginx.
- Configure a sample web page.
- Start and enable the service.
- Verify that the page is accessible.
- Demonstrate repeat execution without unnecessary changes.

Expected Deliverables

- Ansible inventory and playbook.
- Web server configuration.
- Working web page.
- Execution logs.
- README with setup instructions.

Final Demonstration

Execute the playbook, access the web page and run the playbook again to demonstrate idempotency.

Suggested tools: Ansible, Ubuntu, Nginx.

Problem 13: Automated Cloud Storage Backup and Restore

Difficulty: Beginner–Intermediate

Problem Statement

Develop an automated backup and restoration system that stores timestamped copies of selected files and verifies that the files can be recovered without data corruption.

Minimum Requirements

- Select a set of sample files.
- Create timestamped backups.
- Store backups separately from the original files.
- Generate checksums.
- Implement a restoration script.
- Verify restored file integrity.

Expected Deliverables

- Backup script.
- Backup storage directory or object storage.
- Timestamped backup files.
- Restoration script.
- Backup logs and integrity verification results.

Final Demonstration

Back up sample files, remove a working copy, restore the files and verify that their checksums match the originals.

Suggested tools: Python or Bash, MinIO or a designated backup directory.

Problem 14: Infrastructure Configuration Drift Detector

Difficulty: Intermediate

Problem Statement

Build a tool that compares a declared infrastructure configuration with the observed configuration and reports differences without automatically modifying the environment.

Minimum Requirements

- Define a baseline configuration using YAML or JSON.
- Collect the actual state of a test environment.
- Compare expected and observed values.
- Identify mismatched settings.
- Report each discrepancy.
- Recommend a corrective action.

Expected Deliverables

- Baseline configuration.
- State-collection script.
- Comparison engine.
- Drift report.
- Test cases for matching and mismatching configurations.

Final Demonstration

Modify a monitored setting, run the checker and demonstrate that it identifies the configuration drift.

Suggested tools: Python, YAML, JSON, Ansible facts or Linux configuration utilities.

Problem 15: Cloud Resource Cost Estimator

Difficulty: Beginner–Intermediate

Problem Statement

Develop a tool that estimates infrastructure costs using a predefined pricing dataset and compares alternative infrastructure configurations.

Minimum Requirements

- Provide a documented resource-pricing dataset.
- Allow users to select resource types and quantities.
- Calculate estimated hourly and monthly costs.
- Compare at least two configurations.
- Display a cost breakdown.
- Identify potential cost savings.

Expected Deliverables

- Pricing dataset.
- Input interface or command-line tool.
- Cost-calculation engine.
- Comparison report.
- README documenting assumptions and limitations.

Final Demonstration

Compare two infrastructure configurations and explain which resources contribute most to their cost differences.

Suggested tools: Python or JavaScript, CSV or JSON, Streamlit or a simple web interface.

Scope restriction: Actual cloud billing integration is not required.

Category 4: Monitoring, Logging and Reliability

These challenges focus on service monitoring, incident detection, centralized logging and operational reporting.

Problem 16: Real-Time Server Health Dashboard

Difficulty: Beginner–Intermediate

Problem Statement

Develop a dashboard that displays server CPU utilization, memory usage, disk usage and uptime, helping administrators identify resource problems.

Minimum Requirements

- Collect metrics from a Linux server.
- Display CPU, memory and disk usage.
- Display uptime or server availability.
- Configure at least two warning thresholds.
- Show changing metrics under a controlled workload.
- Deploy the dashboard to an accessible server.

Expected Deliverables

- Metrics-collection component.
- Dashboard.
- Configurable thresholds.
- Historical or periodically updated metrics.
- README and demonstration evidence.

Final Demonstration

Generate CPU or memory load and show the corresponding changes on the dashboard.

Suggested tools: Python, Flask, psutil, Prometheus and Grafana.

Problem 17: Alerting and Incident Notification System

Difficulty: Beginner–Intermediate

Problem Statement

Build a monitoring system that detects service failures and generates notifications containing the affected service, detected condition and time of occurrence.

Minimum Requirements

- Monitor a service or application endpoint.
- Configure two alert conditions.
- Generate notifications through email, a webhook or a local receiver.
- Include useful incident information in each alert.
- Detect service recovery.
- Record alert events.

Expected Deliverables

- Monitoring configuration or script.
- Two alert rules.
- Notification mechanism.
- Alert and recovery logs.
- README with testing instructions.

Final Demonstration

Stop a monitored service, demonstrate an alert, restart the service and show that recovery is detected.

Suggested tools: Python, HTTP health checks, Prometheus and Alertmanager, or a local webhook receiver.

Problem 18: Centralized Log Search Platform

Difficulty: Intermediate

Problem Statement

Develop a centralized logging solution that collects logs from at least two application services and allows users to search and filter them through a common interface.

Minimum Requirements

- Create or use two sample services.
- Collect logs from both services.
- Store or aggregate the logs centrally.
- Filter logs by service and severity.
- Search logs by message content.
- Demonstrate investigation of a sample error.

Expected Deliverables

- Two log-producing services.
- Log-collection configuration.
- Centralized log interface.
- Example error queries.
- README and test evidence.

Final Demonstration

Generate errors in both services and retrieve the relevant logs from the same interface.

Suggested tools: Grafana Loki, Grafana, Docker Compose.

Scope restriction: Use predefined logging configurations where possible. Building a complete production logging platform is not required.

Problem 19: Automated Root-Cause Analysis Assistant

Difficulty: Intermediate

Problem Statement

Develop a diagnostic assistant that analyzes application logs and health-check results to identify likely causes of common operational failures and recommend investigation steps.

Minimum Requirements

- Accept logs or structured incident data.
- Implement rules for at least four failure scenarios.
- Identify the likely cause of each scenario.
- Display the evidence supporting each diagnosis.
- Recommend troubleshooting steps.
- Test the assistant against multiple incident examples.

Expected Deliverables

- Diagnostic engine.
- Sample incident datasets.
- Rule definitions.
- Diagnostic reports.
- Test results and documentation.

Final Demonstration

Provide different incident datasets and demonstrate how the assistant produces diagnoses supported by the input evidence.

Suggested tools: Python, regular expressions, JSON.

Scope restriction: A rule-based solution is sufficient. An LLM integration is optional.

Problem 20: Service Availability and SLA Reporter

Difficulty: Beginner–Intermediate

Problem Statement

Build a monitoring tool that periodically checks a service endpoint and calculates its availability, downtime and response-time statistics.

Minimum Requirements

- Accept a configurable HTTP endpoint.
- Perform periodic health checks.
- Record timestamps and response statuses.
- Calculate availability percentage and observed downtime.
- Calculate average response time.
- Export a CSV report.

Expected Deliverables

- Monitoring script or service.
- Timestamped monitoring records.
- Availability calculations.
- CSV report.
- README explaining the calculation method.

Final Demonstration

Run successful and failed health checks against a test service and generate a report showing the resulting availability.

Suggested tools: Python, SQLite, HTTP client libraries.

Category 5: DevSecOps and Security Automation

These challenges focus on vulnerability detection, secret protection, infrastructure compliance and software integrity.

Problem 21: Automated Container Vulnerability Scanner

Difficulty: Intermediate

Problem Statement

Create an automated workflow that scans container images for known vulnerabilities and prevents images exceeding a configured severity threshold from proceeding to deployment.

Minimum Requirements

- Select a suitable training image.
- Scan the image using a vulnerability scanner.
- Generate a report of detected vulnerabilities.
- Configure a severity threshold.
- Integrate the scanner into a CI workflow or equivalent automated pipeline.
- Demonstrate a policy-triggered failure.

Expected Deliverables

- Scanning configuration.
- Automated scanning workflow.
- Vulnerability report.
- Severity policy.
- Evidence of a passing and failing scan.

Final Demonstration

Scan an appropriate training image, demonstrate the configured policy and show how an updated image changes the result.

Suggested tools: Trivy, Docker, GitHub Actions.

Problem 22: Secret Leak Detection in Git Repositories

Difficulty: Beginner–Intermediate

Problem Statement

Develop a secret-detection workflow that identifies dummy credentials accidentally introduced into source code and prevents the affected change from passing the configured security checks.

Minimum Requirements

- Create a sample Git repository.
- Configure a secret scanner.
- Include safe dummy secret examples.
- Integrate scanning into a pre-commit hook or CI workflow.
- Report the affected file and detected issue.
- Demonstrate detection of a dummy credential.

Expected Deliverables

- Sample repository.
- Secret-scanning configuration.
- Pre-commit hook or CI workflow.
- Detection report.
- README explaining safe secret handling and remediation.

Final Demonstration

Attempt to commit a dummy API key and demonstrate that the scanner detects or rejects the change.

Suggested tools: Gitleaks, Git hooks, GitHub Actions.

Problem 23: Infrastructure Security Compliance Checker

Difficulty: Intermediate

Problem Statement

Develop a security-checking workflow that evaluates Infrastructure as Code configurations against a small set of predefined security policies.

Minimum Requirements

- Create compliant and noncompliant sample configurations.
- Define at least three security policies.
- Scan the configurations automatically.
- Identify policy violations.
- Configure a validation gate that rejects a critical violation.
- Demonstrate a correction.

Expected Deliverables

- Sample infrastructure configurations.
- Security policy definitions.
- Automated scanning workflow.
- Violation report.
- Evidence of successful validation after correction.

Final Demonstration

Scan a configuration with an overly permissive access rule, demonstrate the violation, correct the configuration and rerun the checker.

Suggested tools: Checkov or OPA/Conftest, Terraform, GitHub Actions.

Problem 24: Automated Linux Security Baseline Auditor

Difficulty: Intermediate

Problem Statement

Build a read-only auditing tool that evaluates a Linux machine against a small set of security baseline checks and recommends corrective actions for failed checks.

Minimum Requirements

- Implement at least five security checks.
- Inspect relevant Linux settings.
- Report passed, failed and unverified checks.
- Explain why each failed check matters.
- Recommend remediation.
- Avoid automatically modifying system settings.

Expected Deliverables

- Security-auditing script.
- Baseline definitions.
- Audit report.
- Remediation recommendations.
- README documenting supported checks.

Final Demonstration

Run the auditor against a test Linux machine and display the results of at least three checks.

Suggested tools: Bash or Python, OpenSSH configuration, systemd, Linux file permissions.

Problem 25: Software Supply Chain Verification

Difficulty: Beginner–Intermediate

Problem Statement

Develop an artifact-integrity verification workflow that calculates a checksum during a build and verifies the artifact before deployment.

Minimum Requirements

- Create a sample artifact.
- Generate a SHA-256 checksum.
- Store the checksum alongside artifact metadata.
- Implement a verification script.
- Verify an unchanged artifact successfully.
- Detect a modified artifact.

Expected Deliverables

- Sample artifact.
- Checksum-generation script.
- Metadata manifest.
- Verification script.
- Test evidence for successful and failed verification.

Final Demonstration

Verify an untouched artifact, modify a copy and demonstrate that the altered artifact fails verification.

Suggested tools: Python or Bash, SHA-256, GitHub Actions.

Category 6: Networking and Infrastructure Operations

These challenges focus on connectivity diagnostics, traffic routing, DNS, network configuration and firewall policies.

Problem 26: Automated Network and Service Diagnostics

Difficulty: Beginner–Intermediate

Problem Statement

Develop a diagnostic tool that checks network and application connectivity and provides troubleshooting suggestions based on the observed results.

Minimum Requirements

- Accept a hostname, IP address or service endpoint.
- Check DNS resolution.
- Check TCP port connectivity.
- Check HTTP response status and response time.
- Display individual pass/fail results.
- Provide troubleshooting suggestions for failed checks.

Expected Deliverables

- Diagnostic CLI or web application.
- Network-checking functions.
- Structured diagnostic report.
- Test cases covering healthy and failing endpoints.
- README with execution instructions.

Final Demonstration

Test a working service and a controlled failing service, then compare the diagnostic results.

Suggested tools: Python, socket, `dig`, HTTP client libraries.

Problem 27: Reverse Proxy and Load Balancer

Difficulty: Intermediate

Problem Statement

Deploy two backend application instances behind a reverse proxy that distributes incoming requests and continues serving requests when one backend becomes unavailable.

Minimum Requirements

- Create two distinguishable backend instances.
- Configure Nginx as a reverse proxy.
- Implement a load-balancing strategy.
- Identify which backend handled each request.
- Test the system with one backend unavailable.
- Make the application accessible externally.

Expected Deliverables

- Reverse-proxy configuration.
- Two backend instances.
- Load-balancing demonstration.
- Failure-test results.
- Architecture diagram and README.

Final Demonstration

Send repeated requests, demonstrate distribution across both backends, stop one backend and verify that requests continue reaching the available backend.

Suggested tools: Nginx, Docker Compose, Python or Node.js.

Problem 28: Local DNS Management and Health Checker

Difficulty: Intermediate

Problem Statement

Create a DNS validation tool that compares actual DNS resolution results with a predefined list of expected hostnames and IP addresses.

Minimum Requirements

- Define at least three expected DNS records.
- Configure a local DNS service or use a predefined DNS test environment.
- Query the configured DNS records.
- Compare actual and expected results.
- Identify missing or mismatched records.
- Generate a validation report.

Expected Deliverables

- DNS configuration or test dataset.
- Expected-record definitions.
- DNS validation script.
- Report identifying valid and invalid records.
- README with setup and testing instructions.

Final Demonstration

Validate several records, introduce an incorrect mapping in the test environment and demonstrate that the checker detects the mismatch.

Suggested tools: CoreDNS or BIND9, Python, `dig`.

Scope restriction: A predefined DNS environment or simulated record dataset is acceptable if local DNS configuration becomes an environmental obstacle.

Problem 29: Network Configuration Drift Monitor

Difficulty: Intermediate

Problem Statement

Develop a tool that compares the current network configuration of a Linux machine with an approved baseline and reports unexpected changes.

Minimum Requirements

- Capture expected interface addresses, routes and DNS settings.
- Collect the current network state.
- Compare actual and expected values.
- Identify mismatches.
- Generate a structured drift report.
- Avoid automatically modifying network settings.

Expected Deliverables

- Baseline configuration.
- Network-state collector.
- Comparison script.
- Drift report.
- Test evidence and documentation.

Final Demonstration

Capture the baseline, change a test setting and run the checker to identify the difference.

Suggested tools: Python, Linux `iproute2`, JSON.

Problem 30: Firewall Rule Audit and Generator

Difficulty: Intermediate

Problem Statement

Build a tool that audits a declared firewall policy, identifies overly permissive rules and generates a proposed corrected configuration for review.

Minimum Requirements

- Accept a firewall policy in JSON or YAML.
- Define at least three validation rules.
- Identify unnecessarily broad permissions.
- Report each violation and its severity.
- Generate a proposed corrected configuration.
- Avoid automatically applying firewall changes.

Expected Deliverables

- Firewall policy input.
- Validation rules.
- Audit report.
- Generated candidate configuration.
- Test cases and README.

Final Demonstration

Audit a sample policy containing unnecessary access, show the findings and generate a more restrictive configuration.

Suggested tools: Python, YAML or JSON, UFW or nftables in a test environment.

Category 7: Developer Productivity and Operational Automation

These challenges focus on developer setup, deployment approvals, backup recovery, performance benchmarking and self-service deployment.

Problem 31: Automated Development Environment Setup Wizard

Difficulty: Beginner–Intermediate

Problem Statement

Develop a setup tool that checks prerequisites, prepares a sample application's environment and launches the application with minimal manual configuration.

Minimum Requirements

- Define the application's prerequisites.
- Check whether required tools are installed.
- Create a configuration file from a safe template.
- Report missing requirements.
- Provide a repeatable setup or launch command.
- Avoid overwriting existing configuration without permission.

Expected Deliverables

- Sample application.
- Prerequisite checker.
- Setup script.
- Configuration template.
- Working application and README.

Final Demonstration

Run the setup workflow in a prepared test environment and demonstrate that it checks prerequisites and launches the application.

Suggested tools: Python or Bash, Git, Docker Compose.

Problem 32: Deployment Approval and Audit Portal

Difficulty: Intermediate

Problem Statement

Develop a lightweight portal that records deployment requests, approval decisions, version identifiers and deployment outcomes.

Minimum Requirements

- Create a deployment-request interface.
- Store requests in a persistent database.
- Implement pending, approved and rejected states.
- Record timestamps and request details.
- Trigger a predefined deployment script or a safe simulated deployment.
- Display the deployment outcome.

Expected Deliverables

- Web interface.
- Backend API.
- Persistent database.
- Approval workflow.
- Audit history.
- README and demonstration data.

Final Demonstration

Submit a deployment request, approve it, trigger the predefined workflow and inspect the resulting audit record.

Suggested tools: FastAPI or Flask, SQLite, HTML/CSS, Docker.

Problem 33: Database Backup Verification and Recovery

Difficulty: Intermediate

Problem Statement

Develop an automated workflow that backs up a sample database, restores the backup into a separate test database and verifies that the restored data matches the original.

Minimum Requirements

- Create a sample database.
- Generate a timestamped backup.
- Validate the backup file.
- Restore it into a separate test database.
- Compare record counts or selected records.
- Generate a verification report.

Expected Deliverables

- Sample database and schema.
- Backup script.
- Restoration script.
- Verification mechanism.
- Logs and test report.

Final Demonstration

Create a backup, restore it into a clean test database and demonstrate that the restored records match the original dataset.

Suggested tools: MySQL or PostgreSQL, Bash or Python, Docker.

Problem 34: Deployment Performance Benchmarking Tool

Difficulty: Intermediate

Problem Statement

Build a benchmarking tool that measures and compares the performance of two application versions under the same workload.

Minimum Requirements

- Prepare two distinguishable application versions or configurations.
- Generate a controlled request workload.
- Measure response time.
- Measure throughput and error rate.
- Generate a comparison report.
- Document the test conditions.

Expected Deliverables

- Two application versions or configurations.
- Load-testing script.
- Performance measurements.
- Comparison report with charts or a table.
- README explaining the methodology.

Final Demonstration

Run the same workload against both versions and display the resulting performance differences.

Suggested tools: k6 or Locust, Python, CSV, Matplotlib.

Problem 35: Self-Service DevOps Deployment Portal

Difficulty: Intermediate–Advanced

Problem Statement

Develop a web portal through which an authorized user can select a predefined application, choose a supported configuration and initiate a deployment workflow.

Minimum Requirements

- Create a web interface for selecting one application.
- Provide two or three predefined configuration options.
- Validate deployment requests on the backend.
- Trigger a predefined deployment script.
- Display queued, running, successful or failed status.
- Display the deployed application’s endpoint or a useful failure message.

Expected Deliverables

- Web interface and backend.
- Predefined deployment script.
- Status-tracking mechanism.
- Deployment logs.
- Working deployed application.
- README documenting the workflow.

Final Demonstration

Select the sample application, initiate deployment through the portal, monitor the deployment status and open the resulting application.

Suggested tools: FastAPI or Flask, Docker Compose, SQLite.

Scope restriction: Support only one application and a small set of predefined configurations. Do not implement arbitrary shell-command execution or a general-purpose deployment platform.

Common Submission Requirements

Every team must submit the following, regardless of the selected problem statement.

Deliverable	Requirement
Source code	Repository containing application code, configuration and scripts.
Live deployment	An externally accessible application or operational endpoint.
README	Problem summary, prerequisites, execution steps and deployment instructions.
Architecture diagram	A simple diagram showing the components and their communication.
Test evidence	At least one meaningful test demonstrating the primary functionality.
Demonstration	A short explanation and live demonstration of the working solution.

Teams should document the actual implementation and its limitations. A working minimum implementation is preferable to incomplete additional features.

Recommended Judging Criteria

Use a common rubric across all 35 problems to make the evaluation fair.

Criterion	Weight
Functional correctness	30%
Deployment and external accessibility	25%
DevOps implementation	20%
Testing and failure handling	15%
Documentation and explanation	10%
Total	**100%**

Judges should verify the core functionality and deployment independently. A deployed application that does not implement its assigned problem should not receive full marks.

Final Recommendation Before Publishing

The revised list is designed for your actual schedule, but three operational decisions remain important.

1. Confirm the deployment environment. If teams must independently obtain cloud accounts, configure infrastructure and troubleshoot permissions during the event, even the easier problems become riskier. Require account and access verification before the event.

2. Decide whether participants must use public cloud infrastructure. Your current announcement permits self-hosted servers and VPS environments, so I have preserved that flexibility. Docker-based deployment on an externally accessible Linux server is sufficient for many challenges.

3. Publish the scope restrictions along with the problems. In particular, Problem 8 requires a prepared Kubernetes environment, Problem 18 benefits from predefined logging configurations, and Problem 35 must remain limited to one application.

With those conditions in place, you can retain all 35 problem statements and allow the 30 teams to select their preferred challenges without requiring every team to build a production-grade platform in a few hours.